A Vanilla zkVM Architecture

Succinct Labs

April 23, 2026

This document is a reference example illustrating the expected scope and level of rigor. It is not meant to describe a zkVM that is implementable and efficient, and the described zkVM may not even be sound. Submitted documents should provide additional detail where applicable.

Overview

This document describes a sample *vanilla zkVM* architecture designed according to the W1 deliverable of the EF zkVM whitepaper guidelines[1]. The architecture proves execution of a fixed RV+ (extended RISC-V) guest program using:

- Segmented execution traces
- Specialized constraint systems ("chips")
- A system of interaction buses
- LogUp arguments for buses consistency
- WHIR-based STARK proofs
- Recursive aggregation of segment proofs

After execution, the trace is divided into segments. Each segment is proven by four inner circuits tied together by a shared bus, whose proofs are verified by a single segment circuit. Segment proofs are then recursively aggregated in a binary tree until a single proof attests to correctness of the entire program execution.

1 Execution Model

We define how a program is executed.

1.1 Program, Execution, and VM State

A **program** $\mathcal{P}$ is a fixed RV+ binary, compiled from an Ethereum State Transition function. Its inputs $\mathcal{I}$ are derived from the Ethereum state (*initial state*), and the execution witness $\mathcal{W}$, and its outputs $\mathcal{O}$ are the final Ethereum state (*final state*).

The program $\mathcal{P}$ operates on the **VM state** S, defined as concatenation of the following components:

[1] `https://zkevm.ethereum.foundation/blog/cryptography-research-update`

- $\texttt{pc}$: 32-bit program counter
- $\texttt{regs}[0..k-1]$: list of k 32-bit register values
- $\texttt{mem}[]$: byte-addressed writable memory

We separately denote the code of the program by $\texttt{code}$. It is immutable and read-only, and is not part of the VM state.

A single instruction step of the program execution is defined as a transition from one VM state to another according to the semantics of the instruction op retrieved from $\texttt{code}$ at the program counter $\texttt{pc}_1$. This is denoted as follows:

$$S := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \xrightarrow{\text{op}:=\texttt{code}[\texttt{pc}_1]} S' := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2) \tag{1}$$

For example, if op is an addition instruction, which adds the first two registers and stores in the third register, then the semantics of the step is defined as

$$\texttt{pc}_2 = \texttt{pc}_1 + 1, \tag{2}$$
$$\texttt{regs}_2 = \texttt{regs}_1 \text{ except that } \texttt{regs}_2[2] := \texttt{regs}_1[0] + \texttt{regs}_1[1], \tag{3}$$
$$\texttt{mem}_2 = \texttt{mem}_1. \tag{4}$$

We will use the notation $(\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \to (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ to denote that S transitions to S' by executing instruction $\text{op} := \texttt{code}[\texttt{pc}_1]$.

An **execution trace** for program $\texttt{code}$ applied to inputs $\mathcal{I}$ is a sequence of VM states

$$\mathcal{T}: \quad S_0 \to S_1 \to \cdots \to S_T$$

where each step corresponds to execution of one instruction.

A **zkVM proof** for $(\mathcal{P}, \mathbb{E}, \mathbb{E}')$, where $\mathbb{E}$ and $\mathbb{E}'$ are the initial and final Ethereum states, is a SNARK that proves the following:

$$\begin{cases} S_0 \to \cdots \to S_T; \\ S_0 := (\texttt{pc}_0 := 0, \texttt{regs}, \texttt{mem}) \text{ where } \texttt{regs}, \texttt{mem} \text{ are derived from } \mathbb{E}; \\ S_T := (\texttt{pc}_T, \texttt{regs}', \texttt{mem}') \text{ where } \texttt{regs}', \texttt{mem}' \text{ are derived from } \mathbb{E}'; \end{cases}$$

1.2 Supported Instructions

The zkVM supports the RV+ instruction set, which is an extension of the RV32IM instruction set. Concretely, all operations:

- conform to the state transition (1), i.e. transform $(\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ to $(\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ according to the semantics of the instruction op retrieved from $\texttt{code}$ at the program counter $\texttt{pc}_1$.
- are all standard RISC-V instructions[2] (executed according to the standard RISC-V semantics) or additional instructions (precompiles) for efficient execution of certain operations — concretely, Keccak and Poseidon.

[2] https://riscv.org/technical/specifications/

For our purpose the exact semantics of the instructions is not important, and we treat them as black boxes with their own predicates modeling the correctness of the execution.

Concretely, for given instruction *op*, the predicate φ_{op} has the semantics

$$\varphi_{op}(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1, \mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) = \text{TRUE} \iff (\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1) \xrightarrow{\text{op}} (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) \wedge \mathtt{code}[\mathtt{pc}_1] = op \quad (5)$$

For instance, if *op* is an ADD instruction, then $\mathtt{mem}_1 = \mathtt{mem}_2$, $\mathtt{pc}_1 + 1 = \mathtt{pc}_2$ and the registers change according to the ADD instruction.

Exceptions are memory operations, which we assume have the following syntax and semantics:

$$\text{read}: \ (\mathtt{regs}_1 := (addr, *, \ldots), \mathtt{mem}) \to (\mathtt{regs}_2 := (addr, x, \ldots), \mathtt{mem}): \qquad \mathtt{mem}[addr] = x, \quad (6)$$

$$\text{write}: \ (\mathtt{regs}_1 := (addr, x, \ldots), \mathtt{mem}) \to (\mathtt{regs}_2 := (addr, x, \ldots), \mathtt{mem}'): \qquad \mathtt{mem}'[i] = \begin{cases} x & \text{if } i = addr, \\ \mathtt{mem}[i] & \text{otherwise.} \end{cases} \quad (7)$$

2 Segmentation

We now explain how execution is split into segments.

2.1 Execution Segments

After execution, the execution trace $\mathcal{T}$ is divided into segments $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_m$ of exactly N_{seg} instructions[3] each (cf. Fig. 1). Each segment $\mathcal{G}_i$ corresponds to a contiguous subsequence of the execution trace, from S_{start_i} to S_{end_i}, and is proven by a set of separate proofs. The segmentation of the execution trace into consecutive segments allows reducing the proof of the entire execution to proving the validity of each segment and consistency between them. The global consistency argument asserts that the segments are consistent with each other and with the public inputs/outputs:

$$S_{end_i} = S_{start_{i+1}} \text{ for all } i \in [m-1], \quad S_{start_1} = S_0, \quad S_{end_m} = S_T.$$

2.2 Segment layout

Each segment is represented in an equivalent form as a collection of one trace and several *chips*, listed below (cf. Fig. 2).

Segment trace It consists of all operations of the execution trace within the segment and state variables, including the program counter, register values, memory state, etc. It is a collection of N_{seg} rows, each corresponding to one instruction step from the segment. The segment trace does not enforce the semantics of all instructions, but only a subset of them, and the rest of the instructions are proven in separate chips and their consistency with the segment trace is enforced by *consistency arguments*.

Keccak chip It consists of all Keccak precompile calls in the segment, and their inputs/outputs.

[3]The instruction set has a no-op instruction to pad the segment to the full length. The parameter N_{seg} is assumed to be hardcoded and fixed, and known to the verifier.

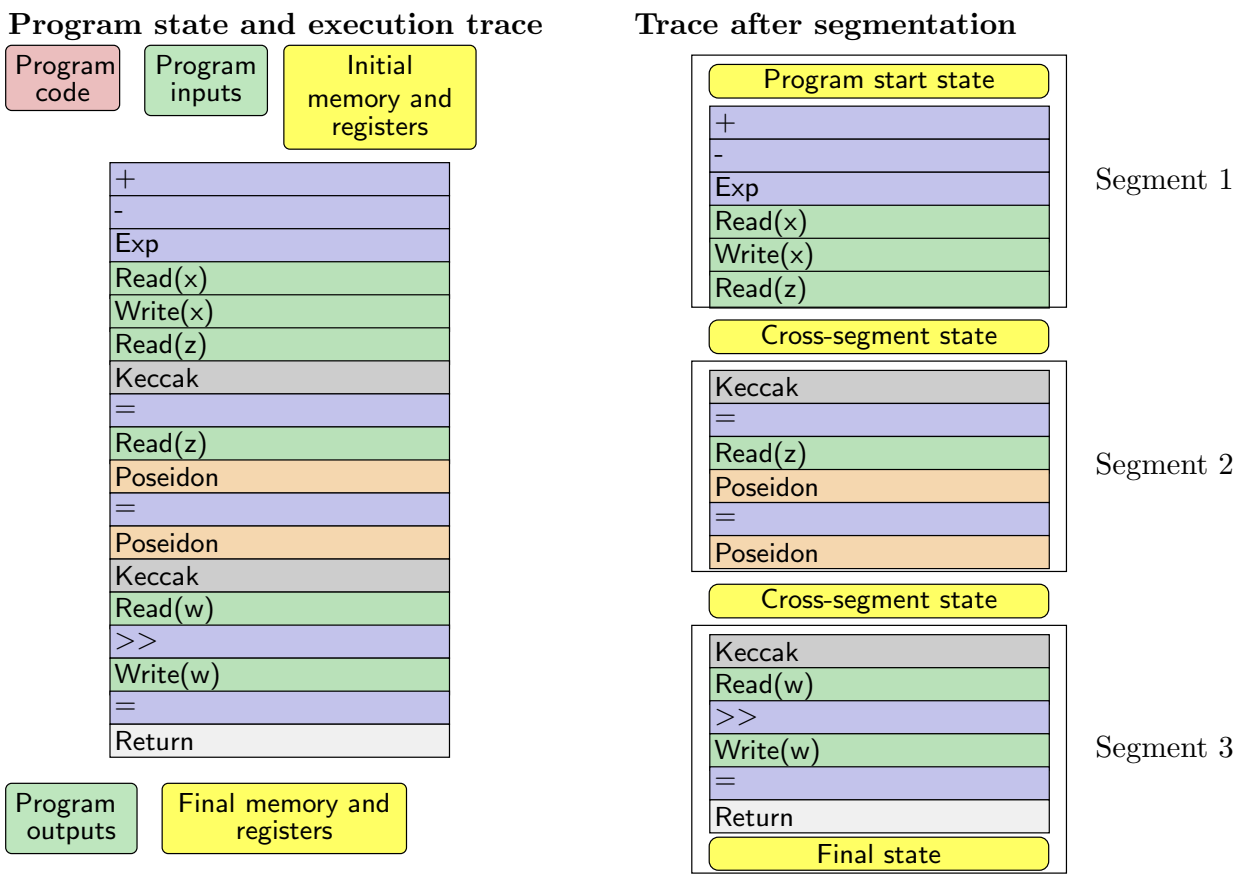

Figure 1: Segmentation of the execution trace into 3 segments

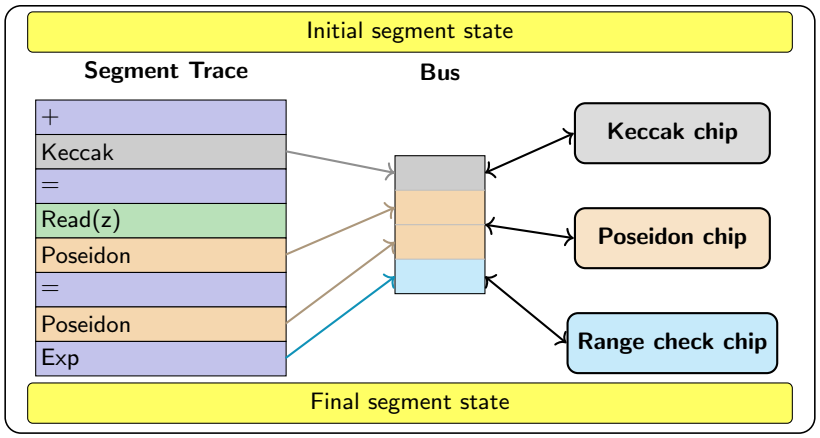

Figure 2: Segment structure: segment trace connects to the bus which then connects to chips.

Poseidon chip It consists of all Poseidon precompile calls in the segment, and their inputs/outputs.

Range check chip It consists of all range check statements occurring in the segment.

Local consistency argument

Teams should describe how their bus consistency argument works in terms of lookups used and how they integrate into the overall proof system.

The consistency arguments enforce that the operations proven in the chips are consistent with the segment trace. This is done using the concept of a *bus* connecting the segment trace and the chips, and lookup arguments to enforce consistency of the bus values with the segment trace and the chip traces.

3 Correct execution of a program

In this section, we define via predicates what it means that a program is correctly executed. We will use these predicates in Section 4 to define the relations that the zkVM proves.

3.1 Predicates for individual operations

Teams should list all operations supported by their zkVM and define the corresponding predicates based on their architecture.

Each operation is treated as a black box with its own predicate φ_{op} (cf. (5)). Throughout this section, subscripts 1 and 2 denote states before and after an operation. We also denote $S_1 := (\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1)$ and $S_2 := (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2)$ as the states before and after the operation.

We group operations as follows:

- **Memory operations.** Read: $\varphi_{\text{read}}(S_1, S_2)$ — does not modify memory. Write: $\varphi_{\text{write}}(S_1, S_2)$ — may modify memory.
- **Precompiles.** Keccak: $\varphi_{\text{keccak}}(S_1, S_2)$. Poseidon: $\varphi_{\text{poseidon}}(S_1, S_2)$.

- **Arithmetic without range checks.** $\mathrm{op}_1, \ldots, \mathrm{op}_{10}$ with predicates $\varphi_{\mathrm{op_i}}(S_1, S_2)$.
- **Arithmetic with range checks.** $\mathrm{op}_{11}, \ldots, \mathrm{op}_{20}$ with predicates $\varphi_{\mathrm{op_i}}(S_1, S_2)$, each decomposable as $\varphi'_{\mathrm{op_i}}(S_1, S_2) \wedge \varphi_{\mathrm{range}}(S_1)$ where the constants defining the range check are always in the last two registers.

All operations except read and write do not read or write memory, so we may omit these arguments from their predicates in the future text for brevity.

3.2 Correct execution of a single step

Teams should define the step predicate for their architecture, specifying how the correct operation is selected and verified at each step.

A single step executes the instruction at the current program counter. The step predicate selects the matching operation via a disjunction over all opcodes:

$$\varphi_{step}(S_1, S_2) := \bigvee_{op \in \mathrm{operations}} \varphi_{op}(S_1, S_2)$$

where $S_1 := (\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1)$ and $S_2 := (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2)$ are the states before and after the step.

3.3 Delegating expensive operations using a Bus

Teams should describe how expensive operations (e.g., precompiles, range checks) are delegated to a bus or similar mechanism in their architecture.

Range checks and precompile calls are expensive to prove inline. Instead, we collect them in a shared *bus* $\mathcal{B}$ and verify separately, proving the consistency along the way. The *bus* $\mathcal{B}$ is a collection of all precompile calls and range checks within a segment. That is, it has the following structure:

$$\mathcal{B} = \{(\underbrace{\mathrm{op}^{(i)}}_{\in\{\mathrm{keccak,poseidon}\}}, S_1^{(i)}, S_2^{(i)})\}_{i=1}^{b} \cup \{(\mathrm{range}, S^{(j)})\}_{j=1}^{r}$$

where b is the number of precompile calls and r is the number of range checks.

Now we define auxiliary predicates that filter the bus by operation type and assert correctness of each entry: $\varphi_{\mathrm{keccak}}(\mathcal{B})$ checks all Keccak entries, $\varphi_{\mathrm{poseidon}}(\mathcal{B})$ checks all Poseidon entries, and $\varphi_{\mathrm{range}}(\mathcal{B})$ checks all range-check entries. Concretely:

$$\varphi_{\mathrm{keccak}}(\mathcal{B}) := \bigwedge_{(\mathrm{keccak}, S_1, S_2) \in \mathcal{B}} \varphi_{\mathrm{keccak}}(S_1, S_2)$$

$$\varphi_{\mathrm{poseidon}}(\mathcal{B}) := \bigwedge_{(\mathrm{poseidon}, S_1, S_2) \in \mathcal{B}} \varphi_{\mathrm{poseidon}}(S_1, S_2)$$

$$\varphi_{\mathrm{range}}(\mathcal{B}) := \bigwedge_{(\mathrm{range}, S) \in \mathcal{B}} \varphi_{\mathrm{range}}(S)$$

We now rewrite the step predicate (3.2) to separate inline verification from bus-deferred verification. The new form is equivalent[4]:

$$\varphi_{\mathrm{step}}(S_1, S_2, \mathcal{B}) := \varphi_{\mathrm{step,bus}}(S_1, S_2, \mathcal{B}) \wedge \varphi_{\mathrm{keccak}}(\mathcal{B}) \wedge \varphi_{\mathrm{poseidon}}(\mathcal{B}) \wedge \varphi_{\mathrm{range}}(\mathcal{B}) \tag{8}$$

[4]Formally the new predicate is even stronger as it also asserts the correctness of the operations in $\mathcal{B}$ not listed in the segment. We omit checking for such operations for efficiency.

where

$$\varphi_{\text{step,bus}}(S_1, S_2, \mathcal{B}) := \bigvee_{op \in \{\text{read,write,op}_1,\ldots,\text{op}_{10}\}} (\varphi_{\text{op}}(S_1, S_2)) \bigvee_{op \in \{\text{op}_{11},\ldots,\text{op}_{20}\}} \left(\varphi'_{\text{op}}(S_1, S_2) \wedge (\text{range}, S_1) \in \mathcal{B}\right) \bigvee_{op \in \{\text{Keccak,Poseidon}\}} ((op, S_1, S_2) \in \mathcal{B}) \quad (9)$$

Recall that $S_1 := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ and $S_2 := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$. The predicate $\varphi_{\text{step,bus}}$ separates operations into three categories:

- **Memory ops and arithmetic without range checks** $(\text{read}, \text{write}, \text{op}_1, \ldots, \text{op}_{10})$ are verified inline via their predicates.
- **Arithmetic with range checks** $(\text{op}_{11}, \ldots, \text{op}_{20})$: the arithmetic logic φ_{op} is verified inline, but the range check is deferred by recording (range, S_1) in $\mathcal{B}$.
- **Precompiles** (Keccak, Poseidon) are fully deferred — the step only checks that the call appears in $\mathcal{B}$, while correctness will be verified by $\varphi_{\text{keccak}}(\mathcal{B})$, $\varphi_{\text{poseidon}}(\mathcal{B})$ in (8).

3.4 Committed memory

Teams should describe how memory is committed and how memory operations are verified in their architecture.

The full memory state is too large to pass around directly, so we replace it with the Merkle tree commitment Com. We define a *state with committed memory* $\hat{S} := (\texttt{pc}, \texttt{regs}, \text{Com})$, replacing the full memory in $S := (\texttt{pc}, \texttt{regs}, \texttt{mem})$ with its commitment.

The read operation predicate becomes

$$\widehat{\varphi}_{\text{read}}(\texttt{pc}_1, \texttt{regs}_1, \text{Com}_1, \texttt{pc}_2, \texttt{regs}_2, \text{Com}_2)$$

with $\text{Com}_1 = \text{Com}_2$, and the write operation predicate becomes

$$\widehat{\varphi}_{\text{write}}(\texttt{pc}_1, \texttt{regs}_1, \text{Com}_1, \texttt{pc}_2, \texttt{regs}_2, \text{Com}_2)$$

with additional semantics that the read and write operations are consistent with the commitments.

Replacing memory with commitments in (8) gives the step relation:

$$\widehat{\varphi}_{step}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) := \widehat{\varphi}_{step,bus}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) \wedge \widehat{\varphi}_{\text{keccak}}(\widehat{\mathcal{B}}) \wedge \widehat{\varphi}_{\text{poseidon}}(\widehat{\mathcal{B}}) \wedge \widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) \quad (10)$$

where $\widehat{\mathcal{B}}$ replaces the memory entries with committed memory (which is practically invisible as the bus operations do not use memory), $\widehat{\varphi}_{\text{keccak}}$, $\widehat{\varphi}_{\text{poseidon}}$, $\widehat{\varphi}_{\text{range}}$ are essentially unchanged as those relations do not use memory, and $\widehat{\varphi}_{step,bus}$ are defined analogously to $\varphi_{\text{step,bus}}$ but with all operation predicates replaced by their committed-memory versions:

$$\widehat{\varphi}_{step,bus}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) := \bigvee_{op \in \{\text{read,write,op}_1,\text{op}_2,\ldots,\text{op}_{10}\}} \left(\widehat{\varphi}_{op}(\hat{S}_1, \hat{S}_2)\right) \bigvee_{op \in \{\text{op}_{11},\text{op}_{12},\ldots,\text{op}_{20}\}} \left(\widehat{\varphi}'_{op}(\hat{S}_1, \hat{S}_2) \wedge (\text{range}, \hat{S}_1) \in \widehat{\mathcal{B}}\right) \bigvee_{op \in \{\text{Keccak,Poseidon}\}} \left((op, \hat{S}_1, \hat{S}_2) \in \widehat{\mathcal{B}}\right) \quad (11)$$

Note that (8) and (10) are no longer equivalent (in terms of soundness) as the second one uses committed memory instead of the actual memory.

4 Proof Architecture

We now define relations that are proven using individual SNARKs, and that recursively combine into the final proof.

4.1 Topology

The proof pipeline consists of multiple circuit types.

Teams should add footnotes to each circuit subsection below linking to the corresponding soundcalc entry for that circuit's parameters and security analysis.

Below we expose some aggregate information.

Circuit	**Purpose**	**Relation**
`core` (many shapes)	Core shard proof	$\mathcal{R}_{\text{shard}}^{(C)}$
`normalize`	1-to-1 shape normalization	$\mathcal{R}_{\text{norm}}$
`compose`	N-to-1 aggregation ($1 \leq N \leq 4$)	$\mathcal{R}_{\text{compose}}$
`deferred`	Deferred sub-computation	$\mathcal{R}_{\text{def}}$

- `core`: Proves the correct execution of one shard (a chip cluster instance). The circuit shape varies with the chip cluster type and trace dimensions (Sec. 4.2).
- `normalize`: A 1-to-1 circuit that verifies a single core shard proof and outputs a proof of a fixed shape (`compress_shape`), irrespective of the input shard's shape (Sec. 4.3).
- `compose`: An N-to-1 recursion circuit ($1 \leq N \leq 4$) that aggregates `normalize`, `compose`, and/or `deferred` proofs into a single proof of `compress_shape` (Sec. 4.4).
- `deferred`: A 1-to-1 circuit that verifies a complete proof of a deferred sub-computation and wraps it as a `compress_shape` proof for inclusion in the `compose` tree (Sec. 4.5).

We visualize the recursion topology in Fig. 3. At the leaves, `core` shard proofs (one per chip cluster instance) are each normalized to `compress_shape` by a `normalize` circuit. The resulting proofs—together with any `deferred` proofs—are aggregated by a tree of `compose` circuits (fan-in up to 4). The recursion terminates when all proofs have been aggregated into a single `compose` proof.

Programs vs. verifiers. Because of recursive proof aggregation, each recursion step acts as both a *prover* and a *verifier*: the program executed by the prover contains the verifier for the previous step's proof system. For example, the `compose` program contains the compress verifier, which verifies `normalize`, `compose`, and `deferred` proofs. In the subsections below, when we say "the `compose` circuit verifies a proof," we mean that the compose *program* embeds the compress *verifier* as part of its constraint system.

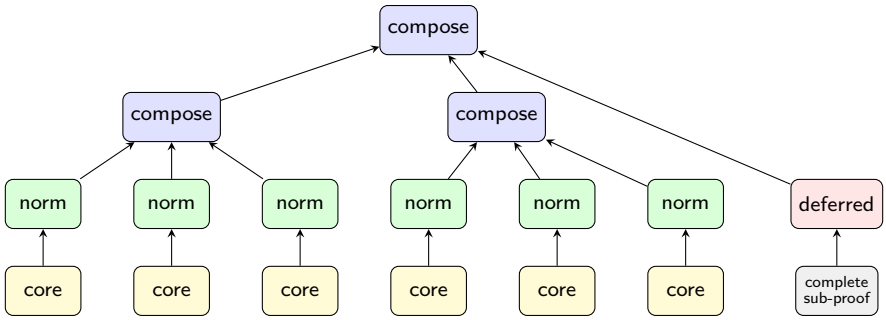

Figure 3: Recursion topology. Colors: core , normalize , compose , deferred . The tree is illustrative; the actual fan-in and depth depend on the number of shards and deferred proofs.

Verifying key Merkle tree. Each recursion circuit type may have multiple *shapes*—distinct circuit configurations that arise from different chip clusters and trace dimensions. A verifying key vk is derived for each shape, yielding approximately 181,000 distinct keys in total. These keys are organized into a Merkle tree whose root vk_root is used to authenticate recursion proofs.

The shapes are enumerated as follows:

- **Normalize** (the vast majority of leaves): one shape per combination of chip cluster, preprocessed and main stacked BaseFold column counts, and padding column counts.
- **Compose**: one shape per arity ($N = 1, 2, 3, 4$).
- **Deferred**: a single shape.

The `compose` program enforces VK authentication: it takes Merkle membership proofs for each input proof's verifying key as part of its witness, verifies them against the Merkle root, and checks that every input proof's rpv.vk_root equals this root. The `deferred` program similarly verifies Merkle membership for its input proof's verifying key. The `normalize` program does not perform VK membership checks itself; it propagates vk_root as a public value, which the compose verifier then checks for consistency.

Note that the reason for the 1-1 normalization step is to enable

4.2 `core` circuits

To optimize the proof size and prover time, we introduce core shard circuits that prove the correctness of RISC-V execution and precompile execution.

The circuits are determined by a "chip cluster," i.e. a collection of chips that are proved together in a single shard. All inter-shard communication is enabled by passing messages through the global accumulation bus, which is arithmetized as a `Global` table present in all shards.

Each chip declares interactions with two possible scopes:

- **Local** interactions must balance within the shard. This is enforced by a LogUp argument (proved via GKR) inside the shard proof.
- **Global** interactions need not balance within a single shard. Instead, each shard computes its net contribution $\sigma \in E(\mathbb{F})$, a point on a degree-7 elliptic curve over the base field (the "septic curve"), and outputs it as a public value. The contributions are accumulated across all shards during recursion, and the final completeness check asserts $\sum_i \sigma_i = \mathcal{O}$ (the identity point).

Each shard proof attests to the relation

$$\mathcal{R}^{(C)}_{\text{shard}} = \left\{ ((\mathsf{pv}, \sigma);\ \mathsf{trace}) \;\middle|\; \begin{array}{c} \text{All chip constraints in cluster } C \text{ are satisfied} \\ \text{All local-scope bus interactions balance (via LogUp)} \\ \sigma \text{ is the correct global cumulative sum for the global-scope interactions} \\ \text{Public values } \mathsf{pv} \text{ are consistent with the trace} \end{array} \right\}$$

where C is the chip cluster determining the circuit, pv are the shard's public values (boundary state, digests, etc.), and σ is the shard's global cumulative sum.

The clusters are of 4 types:

1. **Base core cluster** (`core`). Contains all RISC-V instruction chips (ALU, memory load/-store, control flow, etc.) together with infrastructure chips: `Program` (instruction lookup), `ByteLookup` and `RangeLookup` (preprocessed lookup tables), `MemoryLocal` (local memory state), `MemoryBump` and `StateBump` (timestamp management), `SyscallCore`, `SyscallInstrs` (syscall dispatch), and `Global` (global bus accumulator).

2. **Extended core clusters** (`core+`P). Some precompiles are compact enough to be retained in the core cluster, avoiding the overhead of a separate shard proof and allowing communication with their calling context via the local lookup argument. The retainable extensions are: `MemoryGlobalInit`/`MemoryGlobalFinal`, `Bls12381Fp`, `Bn254Fp`, `Sha256`{`Extend`,`Compress`} (with their control chips), `Uint256Ops`, and `Poseidon2`. Extended clusters are generated for subsets of size 0, 1, or all of these extensions, plus one additional combination (`core` + memory global + SHA-256 + `Uint256Ops`).

3. **Deferred precompile clusters** (`precompile-`P). Precompiles with high arithmetization costs (in terms of number of columns in the arithmetization) are proved in their own shards. Each such cluster contains the precompile-specific chip(s) together with a base set of infrastructure chips: `Program`, `ByteLookup`, `RangeLookup`, `SyscallPrecompile`, `MemoryLocal`, and `Global`. They communicate with their RISC-V calling context via the global lookup argument. The deferred precompiles include: elliptic curve operations (Ed25519, Secp256k1, Secp256r1, BN-254, BLS12-381 point add/double), field arithmetic (BLS12-381 and BN-254 $\mathbb{F}_p/\mathbb{F}_{p^2}$ operations), hash functions (Keccak permutation, SHA-256, Poseidon2), and big-integer arithmetic (`Uint256Mul`, `Uint256Ops`).

4. **Memory boundary cluster** (`mem-boundary`). The global memory argument requires initialization and finalization: sending the initial memory state into the global bus and receiving the final memory state from the global bus. This cluster contains `MemoryGlobalInit`, `MemoryGlobalFinal`, `Global`, and the preprocessed chips. Note that when memory initialization/finalization is retained into an extended core cluster (type 2), this standalone cluster is not needed (this happens for very small programs so that the proof can fit into one shard).

4.3 `normalize` circuit

The `normalize` circuit proves the correct verification of a single core shard proof. Its output is a proof of a fixed shape (`compress_shape`), irrespective of the input shard proof's shape (which varies with the chip cluster). The circuit takes as witness a shard proof π, the verifying key vk, a completeness flag $\mathsf{is_complete} \in \{0, 1\}$, a deferred-proof counter n_{def}, a VK Merkle root $\mathsf{vk_root}$, and a reconstructed deferred-proofs digest $d^{\mathsf{recon}}_{\mathsf{def}}$.

The circuit performs two categories of work:

- **Shard proof verification.** A Fiat–Shamir challenger is initialized by observing the verifying key, and the core shard proof is verified against this challenger (polynomial commitment openings and constraint satisfaction).
- **Aggregation checks.** Outside the shard verifier, the circuit enforces cross-shard and global correctness:
  - *First-shard PC validation.* If the shard is the first execution shard, the circuit asserts that $\mathsf{pc_start}$ matches the verifying key's start PC.
  - *Global cumulative sum accumulation.* The output global cumulative sum σ is computed by adding the verifying key's $\mathsf{initial_global_cumulative_sum}$ (if this is the first shard, otherwise $\mathcal{O}$) to the shard's own $\mathsf{global_cumulative_sum}$.
  - *Public values propagation.* The shard's public values (boundary PCs, timestamps, memory addresses, digests, exit codes, etc.) are propagated into a `RecursionPublicValues` struct together with the VK digest and VK Merkle root. A Poseidon-2 digest of these values is committed as the proof's public output.
  - *Completeness conditions.* When $\mathsf{is_complete} = 1$, the proof is expected to attest to a full program execution rather than a single shard, so final boundary conditions are checked — including that the program has halted, the first shard was included, all "previous" boundary values are at their initial state, and the global cumulative sum equals $\mathcal{O}$ (the identity point on the septic curve).

Formally, let $\Pi^{(C)}_{\text{shard}}$ be the SNARK for $\mathcal{R}^{(C)}_{\text{shard}}$. Write pv for the shard's public values (including its global cumulative sum σ_{shard}) extracted from the proof, and rpv for the recursion public values output by the circuit.

$$\mathcal{R}_{\text{norm}} = \left\{ \mathsf{rpv};\ (\pi,\ \mathsf{vk},\ C) \;\middle|\; \begin{array}{c} \Pi^{(C)}_{\text{shard}}.\mathsf{Verify}(\mathsf{pv},\ \pi) = 1 \\ \mathsf{rpv.contains_first_shard} = 1 \implies \mathsf{pv.pc_start} = \mathsf{vk.pc_start} \\ \mathsf{rpv}.\sigma \ = \ \sigma_{\text{init}} + \mathsf{pv}.\sigma_{\text{shard}}, \quad \sigma_{\text{init}} := \begin{cases} \mathsf{vk}.\sigma_0 & \text{if first shard} \\ \mathcal{O} & \text{otherwise} \end{cases} \\ \mathsf{rpv} \text{ is correctly derived from } \mathsf{pv},\ \mathsf{vk},\ \text{and auxiliary inputs} \\ \mathsf{rpv.is_complete} = 1 \implies \varphi_{\text{complete}}(\mathsf{rpv}) \end{array} \right\}$$

where $\varphi_{\text{complete}}$ asserts the final boundary conditions described above (program halted, first shard included, initial boundary values zero, $\sigma = \mathcal{O}$).

4.4 `compose` circuit

The `compose` circuit is an N-to-1 recursion circuit ($1 \le N \le 4$) that consumes N proofs — each of which may be a `normalize`, `compose`, or `deferred` proof — and produces one proof of the fixed shape `compress_shape`. Its purpose is to aggregate a batch of proofs that cover contiguous portions of the execution into a single proof.

The circuit performs two categories of work:

- **Proof verification.** Each input proof π_i is verified against its own verifying key vk_i. The circuit checks that every proof's $\mathsf{vk_root}$ matches a single witnessed value, and that each proof's recursion public values have a valid digest.

- **Continuity and aggregation checks.** The circuit enforces that the N proofs form a contiguous chain:
  - *Sequential continuity.* For each consecutive pair of proofs, the "last" boundary values of proof i (PC, timestamp, memory addresses, page indices, deferred digest, exit code, syscall flags, etc.) must equal the "previous" boundary values of proof $i+1$.
  - *Program identity.* The $\mathsf{sp1_vk_digest}$ and $\mathsf{proof_nonce}$ are asserted to be identical across all N proofs.
  - *Accumulation.* The global cumulative sums σ_i from all proofs are summed (via elliptic curve addition on the septic curve). The shard counts and $\mathsf{contains_first_shard}$ flags are accumulated.
  - *Completeness conditions.* When $\mathsf{is_complete} = 1$, the same final boundary conditions as in the `normalize` circuit are checked via $\varphi_{\text{complete}}$.

Formally, let Π_{comp} denote the SNARK for `compress_shape` proofs (used by `normalize`, `compose`, and `deferred` alike). Write rpv_i for the recursion public values of the i-th input proof ($1 \le i \le N$, $1 \le N \le 4$), and rpv for the aggregated output.

$$\mathcal{R}_{\text{compose}} = \left\{ \mathsf{rpv};\ (\{\mathsf{vk}_i, \pi_i\}_{i=1}^{N},\ \mathsf{vk_root}) \;\middle|\; \begin{array}{c} \forall\, i:\ \Pi_{\text{comp}}.\mathsf{Verify}(\mathsf{rpv}_i,\ \pi_i) = 1\ \wedge\ \mathsf{rpv}_i.\mathsf{vk_root} = \mathsf{vk_root} \\ \forall\, i < N:\ \mathsf{rpv}_i.\mathsf{last_*} = \mathsf{rpv}_{i+1}.\mathsf{prev_*} \\ \forall\, i:\ \mathsf{rpv}_i.\mathsf{sp1_vk_digest} = \mathsf{rpv}_1.\mathsf{sp1_vk_digest} \\ \mathsf{rpv}.\sigma\ =\ \sum_{i=1}^{N} \mathsf{rpv}_i.\sigma \\ \mathsf{rpv} \text{ boundary values derived from } \mathsf{rpv}_1.\mathsf{prev_*} \text{ and } \mathsf{rpv}_N.\mathsf{last_*} \\ \mathsf{rpv}.\mathsf{is_complete} = 1 \implies \varphi_{\text{complete}}(\mathsf{rpv}) \end{array} \right\}$$

Here "$\mathsf{last_*} = \mathsf{prev_*}$" is shorthand for the continuity of all chained boundary values (PC, timestamp, memory init/finalize addresses, page indices, deferred digest, deferred proof index, exit code, and syscall flags).

4.5 `deferred` circuit

The `deferred` circuit is a 1-to-1 recursion circuit that takes a single supposedly complete SP1 proof (representing a deferred sub-computation), verifies it, and updates the deferred digest. Its output is a proof of the fixed shape `compress_shape` with $\mathsf{is_complete} = 0$, so that it can be composed with execution shard proofs in the `compose` circuit.

The circuit performs the following work:

- **VK Merkle verification.** The input proof's verifying key is checked against a VK Merkle tree, ensuring it belongs to the set of permitted programs.
- **Proof verification and validity.** The input proof π is verified against its verifying key vk. The circuit asserts that the proof is marked $\mathsf{is_complete} = 1$ and $\mathsf{exit_code} = 0$.
- **Deferred digest update.** Given an initial digest d_0, the circuit computes the updated digest
$$d_1 = \text{Poseidon2}\big(d_0 \parallel \mathsf{rpv}'.\mathsf{sp1_vk_digest} \parallel \mathsf{rpv}'.\mathsf{committed_value_digest}\big)$$
  where rpv' denotes the input proof's recursion public values.

- **Output construction.** The output recursion public values have $\mathsf{is_complete} = 0$, $\sigma = \mathcal{O}$, all memory and execution boundary values zeroed, and the deferred proof counter advanced by 1.

Formally:

$$\mathcal{R}_{\text{def}} = \left\{ \mathsf{rpv};\ (\mathsf{vk},\ \pi,\ \mathsf{mkl},\ d_0,\ n_{\mathsf{def}}) \;\middle|\; \begin{array}{l} \Pi_{\text{comp}}.\mathsf{Verify}(\mathsf{rpv}',\ \pi) = 1 \\ \mathsf{rpv}'.\mathsf{is_complete} = 1 \ \wedge\ \mathsf{rpv}'.\mathsf{exit_code} = 0 \\ \mathsf{vk} \text{ verified against } \mathsf{mkl} \\ \mathsf{rpv} \text{ records deferred digest range } [d_0,\ d_1] \\ \mathsf{rpv} \text{ records deferred proof range } [n_{\mathsf{def}},\ n_{\mathsf{def}}{+}1] \\ \mathsf{rpv}.\mathsf{is_complete} = 0, \quad \mathsf{rpv}.\sigma = \mathcal{O} \end{array} \right\}$$

5 Proof system instantiations

Teams should explain how the relations are proven using AIR / IOPs / poly-IOPs and the BCS transform. That is, teams should provide a direct reduction from proving a relation to a low-degree test with FRI or another protocol. The reductions should contain links to the circuits submitted to soundcalc. The purpose of the section is to make explicit how the relations from the previous section are proven.

In the previous sections, we have described how to arithmetize the correct RISC-V execution of a program by decomposing it into a collection of tables of field elements, each of which has algebraic constraints placed on its columns. Furthermore, we allow tables to "send" and "receive" messages with multiplicity through a collection of "interaction kinds". All tables are zero padded to have the same height, call it $\mathcal{N}_{row} = 2^{n_{row}}$. For the RISC-V circuits, we use $n_{row} = 22$. The reason for this padding is that it allows for the verifier to be described as a uniform arithmetic circuit, independent of the unpadded table heights. This naturally introduces some prover and verifier overhead; naively, this can be a very large overhead if one of the tables is wide and short, in which case the padding adds a substantial number of trace cells, but the Hypercube proof system has a number of mitigations to minimize this overhead. We will describe some of these mitigations below, especially because one of them is a cryptographic protocol invented for this particular zkVM context. First, though, let us describe protocol as if it were actually run on the padded tables:

1. Treat each column of each table as a multilinear polynomial in n_{row} variables. Use a batch multilinear polynomial commitment scheme to commit to these polynomials. Let W denote the total number of columns/multilinear polynomials.

2. Use GKR with n_{row} maybe ± 1 rounds to prove the LogUp argument equating the "send" and "receive" multisets across all tables. The output of GKR is a collection of W evaluation claims, one on each column, all at the same point η.

3. Use the point η as the randomness for the zerocheck sumcheck which proves constraint satisfaction. Batch the zerocheck sumcheck with the sumchecks which "prove" the evaluation claims from LogUp+GKR (more precisely, the sumchecks reduce evaluation claims at one point to evaluation claims at another). The output of this batched sumcheck is again a collection of W evaluation claims, not at a new point η'.

4. Use the batch multilinear polynomial commitment scheme to prove these evaluation claims.

If the prover actually materialized the full $W \times \mathcal{N}_{row}$ trace, this would be an enormous quantity of data. For example $W \approx 2^{12}$ is in the mid-range of column counts among various "table clusters" used in SP1. For a 32-bit field such as KoalaBear, this translates into a total of $4 \times W \times \mathcal{N}_{row} = 2^{36}$ bytes, or 64 GB. This is twice the global VRAM of an Nvidia 5090 GPU. Clearly, keeping around the padding is not feasible. In order to mitigate these costs, the SP1 proof system uses the following techniques:

1. Instead of using a batch multilinear PCS directly, we use the jagged polynomial commitment scheme, which is a sparse PCS designed for exactly the kind of sparsity inherent in the table layout as described above. This enables the prover and verifier PCS costs to depend only on the size of the unpadded data (and notably, the verifier PCS costs can be made independent of W). For the internal dense PCS, we use batch BaseFold, which we view as a multilinear PCS for committing to a single multilinear in `log_total_unpadded_trace_area` variables. The evaluation proof for a point $\eta = \eta_n || \eta_k$ consists of

2. For the sumchecks, the contributions of the padding rows are always easy to compute and don't scale with the amount of padding. The prover makes frequent use of this fact.

Important Note: The Hypercube proof system verifier can be described as an arithmetic circuit. The circuit is determined by the total trace area of the unpadded data, and the tables included in the segment. Notably, the circuit is *independent* of the individual tables' unpadded heights. Hence, the total number of verifier circuits can be easily limited to the $\approx 2^{18}$ range, which makes it feasible to build them all instead of using a VM for recursion.

In the following subsections, we describe each section of the protocol in a bit more detail, with an eye towards making the soundness analysis in soundcalc more transparent. Whenever a variable appears in `typewriter font`, it refers to a parameter in the "sp1.toml" file in the soundcalc repository.

5.1 LogUp + GKR Details

TODO: Ask Min about the LogUp+GKR business.

5.2 Zerocheck Details

Recall that we have a trace of height $\mathcal{N}_{row}$ and width W on which we place `num_constraints` algebraic constraints, denoted $C_0, C_1, \ldots, C_{num_constraints}$. Each C_i is a polynomial of total degree at most 3 in W variables, though almost always it depends trivially on most of the variables. It is useful also to write $W = w_1 + \ldots + w_k$, where k is the number of tables in the cluster, and to group the constraints by table, so that $C_{i,j}$ refers to the j-th constraint on the i-th table. Let us write c_i for the number of constraints in the i-th table, and $p_{i,\ell}$ for the ℓ-th column of the i-th table. Finally, let us write h_i for the height of the i-th table (not including padding). The claim for our zerocheck is that, $\forall x \in \{0,1\}^{n_{row}}, \quad i \in [k], \quad j \in [c_i]$:

$$C_{i,j}(p_{i,1}[x], \ldots, p_{i,w_i}[x]) = \begin{cases} 0, & x < h_i, \\ C(0, \ldots, 0), & x \geq h_i \end{cases} \tag{12}$$

Note that the constraint evaluation on zero-padding may not result in 0, so at the moment our protocol does not match exactly with the usual zerocheck. However, note the following. If $\mathrm{geq}_{h_i}(x)$ is the function which, for $x \in \{0,1\}^{n_{row}}$, is zero if $x < h_i$ and 1 if $x \geq h_i$, then its multilinear extension $\widehat{\mathrm{geq}}_{h_i}(z)$ is relatively straightforward to evaluate at arbitrary $z \in \mathbb{F}^{n_{row}}$, using the following

recursion relation. Write $z = (z_0, z_{rest}) \in \mathbb{F} \times \mathbb{F}^{n_{row}-1}$, and write $h_i = h_{i,0} * 2^{n_{row}-1} + h_{i,rest}$. Then, we have:

$$\widehat{\mathrm{geq}}_{h_i}(z) = z_0 \cdot (1 - h_{i,0}) + \widehat{\mathrm{eq}}(z_0, h_{i,0}) \cdot \widehat{\mathrm{geq}}_{h_{i,rest}}(z_{rest})$$

To prove this, first, we sample a random $\zeta \in \mathbb{F}^{n_{row}}$, $\lambda \in \mathbb{F}, \alpha \in \mathbb{F}$, and turn this collection of claims into the single sumcheck claim:

$$\sum_{i=0}^{k-1} \sum_{j=0}^{c_i-1} \lambda^i \alpha^j \widehat{\mathrm{geq}}_{h_i}(\zeta) = \sum_{x \in \{0,1\}^{n_{row}}} \widehat{\mathrm{eq}}_{\zeta}(x) \sum_{i=0}^{k-1} \lambda^i \sum_{j=0}^{c_i-1} \alpha^j C_{i,j}(p_{i,1}[x], \ldots, p_{i,w_i}[x]) \tag{13}$$